CS 4501 · ALPHA DEVELOPMENT

Cleanasaurus Activities Document

The problem, MVP validation status, requirements, AI-assisted test-driven development, and the roadmap to Beta

Team	Nate Kim, Nate Park, Wesley Chang
Repository	github.com/natekim416/cs4501_startup (Alpha merged into `main`: PRs #1, #32–#35)
Issues	29 user stories as GitHub issues #2–#30; 18 closed, 11 open
Metrics measured	November 5, 2026 on `main` at `787454f`
Date	November 5, 2026

Contents

1. **Activity 1 · Hook and MVP hypothesis check**: the problem, the validation design, and placeholder results clearly marked as **unconfirmed**
2. **Activity 2 · Requirement decomposition**: core requirements and LLM integration, personas, 29 user stories with acceptance criteria, release plan
3. **Activity 3 · AI-assisted TDD**: visual engineering metrics, workflow insights, red and green runs, issue-to-test map, review log
4. **Stage & Next Steps**: Alpha milestones, Beta roadmap by week, measurable goals
5. **Appendix**: the Alpha running on a dev-clutter demo folder

Activity 1 · Hook & MVP Hypothesis Check

Source: validation/MVP_VALIDATION.md

Team: Nate Kim, Nate Park, Wesley Chang · **Planned testers:** 10 (5 CS, 5 non-CS) · **Testing window listed in the report:** October 30 – November 2, 2026 (unconfirmed)

THE PROBLEM

"Storage Almost Full."

One developer ran a cleanup tool on their own projects and found **58.68 GB in 128 old `node_modules` folders**. They got 49.89 GB back and kept 9 of the folders. Most of that space rebuilds with one command.

The hard part is knowing which folders are safe. One may hide a `.env` that exists nowhere else. Another may sit in a project with uncommitted work. Get it wrong the night before a deadline and you lose real work, and these folders pile up with every class project and hackathon.

Cleanasaurus lets you say "I'm done with CS 3140," then deletes only what it can prove you can rebuild.

Source: Flavio, "Use npkill to find and delete old node_modules folders," flaviocopes.com/npkill, Oct 2026.

What holds up without the tester data:

- **The problem is documented publicly.** One npkill run found 58.68 GB across 128 `node_modules` folders, and Xcode cleanup guides routinely promise 30–50 GB back.
- **The MVP's limits are in its code.** Its Dino Bin sits on the same drive, so moving files there frees nothing (executor.js:8). It treats git repos and `~/Library` as NEVER zones (profile.js:49, profile.js:11), and its scan skips `node_modules` (scanner.js:6). It couldn't reach developer clutter, whatever testers say.
- **The Alpha now measures this honestly.** FREED is the real change in free disk space (US-12).

The rest of this activity shows the validation design and the reasoning we'd apply. Its numbers are not evidence yet.

> **Changes from the MVP plan:**
>
> - **Repeat use** is measured within 3 days. The MVP plan said within 7 days, so this result can't show whether testers come back after day 3.
> - **First-session approval** was "run a full plan" in the MVP plan. Here it means a tester ran at least 80% of a plan's proposed actions on their own files in session 1. Every action in the preview has a checkbox, so unchecking 2 of 38 (C2) still counts as approving the plan, while running 14 of 20 (C4, 70%) counts as holding back. N3 counts because they ran all 41 actions, even though they undid them afterward. Under the original definition, 3 of 10 testers qualify (30%), which is below the MVP's 40% fail line.
> - **Space freed** is a KPI we added. It wasn't in the MVP plan.

Provisional decision: Pivot. On the placeholder numbers, testers trusted the app (2 of 3 trust KPIs met, and the third close), but it didn't solve a problem they cared about (0 of 2 value KPIs met). We keep the engine and aim it at CS students' rebuildable developer clutter.

1. Running the validation plan

Core hypothesis: Students with cluttered laptops will let an AI clean and reorganize their files, as long as every plan is checked and previewed first.

Target population: 5 CS-related friends and classmates (C1–C5), and 5 non-CS testers (N1–N5): four family members and one non-CS student.

Change from our plan: we planned 15 testers (10 CS, 5 non-CS) and ran 10 (5 CS, 5 non-CS), so the CS results rest on fewer people.

Test 1 (as drafted): A 20-minute think-aloud on the tester's own laptop, with three tasks: free up space, organize files, and scan for malware.

Test 2 (as drafted): 3 days of unsupervised use, measured with the app's opt-in usage log. 8 of 10 opted in and sent us the exported log, and we asked the other two how often they ran it.

What each tester told us (placeholder summaries)

> The percentage is how full each tester's disk was.

Tester	Who	Summary
C1	CS, 94%	Approved all 46 actions in Downloads, but said their space is in the `node_modules` of about 30 projects, which the app won't touch, and they'd run it weekly if it could clean those.
C2	CS, does ML research, 88%	Approved 36 of 38 actions because the app proved the dataset copies were identical, and wants it to clear some models they haven't loaded in a month.
C3	CS, builds iOS apps, 97%	Approved 33 of 35 actions on the Desktop, but had to drag two screenshots back by hand because undo is all-or-nothing, and said their real space is Xcode data the app won't touch.
C4	CS, took the security course, 71%	Ran only 14 of 20 actions after the app flagged their own unsigned app as malware, noticed the Dino Bin frees nothing on disk, and wants a local model because file names are private.
C5	CS (graduate), makes games and music, 89%	Approved 26 of 27 actions, started trusting the app when it asked about a 1 GB drum pack, and wants it to clear Unity `Library` folders that rebuild on open.
N1	Family (dad), 93%	Approved 18 of 30 actions after unchecking every photo move and most wants to ask where a downloaded form went.
N2	Family (mom, least comfortable with technology), 78%	Watched the demo on a fake folder but wouldn't let it touch their own files.
N3	Family (sibling, law student), 96%	Approved a 41-action plan without reading it, undid everything when their essay moved, and wants a one-sentence summary and help removing games they don't play.
N4	Family (sibling, less comfortable with technology), 89%	Approved 20 of 22 actions and loved the clean Desktop, but didn't know what "unsigned" meant and wants scam checks on email attachments.
N5	Friend, pre-med student, 91%	Approved all 33 actions, loved that it remembered which essay draft to keep, and wants it to move last semester's files to an external drive.

2. KPI data (placeholder numbers)

2.1 Quantitative

Trust KPIs ask whether people will let the app act. Value KPIs ask whether it was worth it.

Group	KPI	How we measure it	Target	All (10)	CS (5)	Non-CS (5)	Rating
Trust	First-session approval	Ran at least 80% of a plan's proposed actions on their own files in session 1	≥60%	**7/10 = 70%**	4/5 = 80%	3/5 = 60%	🟢
Trust	Unsafe actions	Shown to a tester, and would lose data, touch a NEVER zone, delete a non-duplicate, or quarantine an unflagged file	0	**0**	0	0	🟢
Trust	Undo rate	Actions undone ÷ actions run in session 1, using the counts in "What each tester told us"	≤10%	**41/267 = 15%**	0/155 = 0%	41/112 = 37%	🟡
Value	Repeat use	Ran another plan within 3 days, on their own	≥40%	**3/10 = 30%**	2/5 = 40%	1/5 = 20%	🟡
Value	Space freed	Median space reported freed ÷ median space wanted back	≥50%	**0.6 ÷ 27.5 GB ≈ 2%**	1.4 ÷ 30 GB ≈ 5%	0.3 ÷ 15 GB = 2%	🔴

Scale: 🟢 met the target · 🟡 close, within 15 points · 🔴 missed by more, or any unsafe action.

Decision rules:

- **Validated:** no trust KPI is 🔴, and every value KPI is 🟢.
- **Pivot:** no trust KPI is 🔴, but a value KPI isn't 🟢. The engine works, but it's aimed at the wrong problem or people.
- **Invalidated:** a trust KPI is 🔴. People won't let the app act, or it isn't safe.

Result: trust 2 of 3 with none 🔴, value 0 of 2, so Pivot. CS testers alone met 3 of 3 and 1 of 2, which is still Pivot. Non-CS testers alone met 2 of 3 with a 🔴 undo rate and 0 of 2, which is Invalidated for that group and one reason the pivot targets CS students.

Tester	Actions kept	Ran ≥80%?	Trust (§2.2)	Reported freed	Wanted back	Rebuildable dev clutter	Test 2 plans
C1	46/46	✅	High	1.4 GB	30 GB	38 GB	0
C2	36/38	✅	High	6.6 GB	40 GB	61 GB	1
C3	33/35	✅	Medium	0.3 GB	50 GB	57 GB	0
C4	14/20	❌ 70%	Low	0.8 GB	15 GB	9 GB	0
C5	26/27	✅	High	2.1 GB	25 GB	27 GB	1
N1	18/30	❌ 60%	Low	0.1 GB	10 GB	—	0
N2	Demo only	❌	Low	—	"no idea"	—	0
N3	0/41, all undone	✅*	Low	0 GB	50 GB	—	0
N4	20/22	✅	Medium	0.6 GB	"no idea"	—	0
N5	33/33	✅	High	0.5 GB	15 GB	—	1

* N3 approved without reading. They ran all 41 actions, so they count as running the plan, then undid all 41, so they kept 0. Without N3, approval is 6/9 = 67%. All 41 undone actions came from N3's single undo, so without N3 the

undo rate is 0%.

Other measurements:

- **Validator:** of 37 plan requests, 36 passed within 3 tries (24 on the first), and 1 fell back to the safe part. It caught 20 errors along the way.
- **Disk:** the FREED counter reported 12.4 GB in total, but **0 bytes were actually freed on disk**, because the Dino Bin is on the same drive.
- **Speed:** a validated plan took a median of 34 seconds.

Where CS testers' space actually is:

Category	Total	Testers who had it
Xcode build data and simulators	57 GB	1
ML model caches (Hugging Face, PyTorch)	44 GB	1
JavaScript packages (`node_modules`, npm cache)	26 GB	2
Docker images and build cache	25 GB	3
Python environments and caches	22 GB	2
Game-engine build folders (Unity `Library/`)	18 GB	1
Total rebuildable	**192 GB** (median 38 GB per tester, range 9–61 GB)	

The same 5 testers had a median of **1.4 GB** of reported savings from duplicates, so their rebuildable clutter is **about 27× larger**.

2.2 Qualitative

Trust level, from what each tester did in session 1:

Level	What we saw	Testers
High	Ran at least 80% of a plan, then asked it to do more or said they'd let it run alone	4: C1, C2, C5, N5
Medium	Trusted the plan but would want to check it every time, or it just wasn't useful to them	2: C3, N4
Low	Held back out of distrust, reversed a plan they hadn't read, or wouldn't use their own files	4: C4, N1, N2, N3

CS testers were much more trusting than non-CS testers: 4 of 5 were Medium or High, against 2 of 5.

Themes. A theme counts if at least 4 of 10 testers, or 3 of the 5 non-CS testers, raised it unprompted.

Theme	Testers	Count
The preview and plain-English reasons are why I approved	C1–C5, N4, N5	7/10
My real space problem is somewhere the app won't go	C1–C5, N1, N3	7/10 (5/5 CS)
Undo and "nothing is gone for good" are why I approved	C1, C2, C5, N4, N5	5/10
It's too technical, or there's too much to read	N2, N3, N4	3/5 non-CS

Below the bar: questions and memory were the best part (3/10), a helper to find files or spot scams (2/10), run by itself (2/10), too slow (2/10), sort by more than file names (1/10), remove things that aren't copies (1/10), and file names are private (1/10).

---

3. Provisional decision: Pivot

> This section reads the placeholder numbers as if they were real. It shows how we'd reason from the results, not a confirmed finding. Real sessions could change the decision.

3.1 What the evidence says

Trust is validated.

- 70% ran a plan in their first session, above our 60% target, and still 67% without N3's weak approval.
- No unsafe action reached anyone, even though the validator caught 20 errors.
- The undo rate is 15%, over our 10% target, and all of it comes from N3's single undo.
- The trust features we designed were the top reasons testers gave for approving: previews with reasons (7/10) and undo (5/10).

Value is not validated.

- Testers wanted a median of 27.5 GB back. The app reported 0.6 GB, and the real number was 0, because the Dino Bin sits on the same disk (executor.js:8).
- Only 30% used it again in Test 2.
- 7 of 10 said, unprompted, that their real space problem was somewhere the app won't go. That's by design today: git repos are NEVER zones (profile.js:49), `~/Library` is too (profile.js:11), and the scan skips `node_modules` (scanner.js:6).

Why not "Validated"? The hypothesis only asked whether people would *let* the app act. They did, but with someone sitting next to them and on low-stakes folders. That shows permission, not demand. An app that frees about 2% of the space people want doesn't have a business, however much they trust it.

Why not just "Invalidated"? Nothing about trust failed. The preview, reasons, validator, undo, and questions are the parts that work, and they carry over. What failed is the problem we aimed them at, so we change the target and keep the engine.

Limits of this evidence. Testers are friends and family, which inflates approval and trust, though value still scored low despite that bias. With n = 10, one tester moves a percentage by 10 points, and by 20 points among the 5 CS testers the pivot leans on, so treat the numbers as directional. Three days is also a short window for a problem that builds up over weeks.

3.2 What we're pivoting to

From: a general file cleaner for "students with cluttered laptops" that sorts Downloads and trashes exact duplicates.

To: Cleanasaurus for developers' laptops, starting with CS students. It reclaims space from developer clutter that can be rebuilt, and it only deletes what the validator can prove you can get back.

This is a **customer-segment pivot** (from everyone with a Mac to CS students) plus a **zoom-in pivot** (one capability, "free up space," becomes the product). The vision stays the same: tell your computer to clean itself, and trust it to do it safely.

New hypothesis:

> CS students whose laptops are over 80% full will, without us in the room, approve a validated plan that actually frees at least 10 GB of rebuildable developer clutter in their first session (≥60% of testers), and at least 40% will run it again within 30 days.
>
> **Moat KPI** *(proposed; team to confirm)*: in a head-to-head with CleanMyMac's purge on their own laptops, at least 50% of those testers prefer Cleanasaurus, and none lose work with it.

What it looks like:

> **Student:** I'm out of space. I'm done with CS 3140 and the fall hackathon, but don't touch my capstone or anything on my resume.
>
> **Dino:** I found 38 GB I can prove you can rebuild:
>
> - `node_modules` in 6 projects you haven't opened since spring: 4.1 GB. `npm ci` rebuilds each in about a minute.
> - 3 Python environments from CS 3140: 2.2 GB. Rebuilt from each `requirements.txt`.
> - Xcode build data for 4 projects that no longer exist: 11 GB. Xcode rebuilds it on the next build.
> - 9 Docker images that no container uses: 14 GB. Re-downloaded on the next `docker pull`.
> - 2 Hugging Face models you haven't loaded since March: 6.7 GB. About 10 minutes each to re-download.
>
> I left `hoohacks-2026` alone because it has changes you never committed.

What stays and what changes:

	MVP	After the pivot
Customer	Students and family with messy laptops	CS students first, then working developers
Problem	Messy Downloads and Desktop	A laptop full of rebuildable dev clutter
What it can remove	Only exact duplicates (SHA-256 proof)	Exact duplicates **and** rebuildable artifacts (rebuild proof)
Where it looks	One folder; skips repos, hidden folders, and `~/Library`	Projects, their ignored build folders, and known tool caches, but never source code
Space freed	Only reported; files sit in the Dino Bin	Real: rebuildables are deleted, and the app shows the actual change on disk
Undo	Undoes the whole last run	Per item; for a rebuildable, one click runs the recorded rebuild command
Why come back	No reason once Downloads is clean	An alert at 85% full, an end-of-semester sweep, and a cleanup after each hackathon
Unchanged	Chat → plan → validator → self-correction → ask when stuck → remember → approve	Same

The rebuild proof (new validator check). An item can be deleted only if all five hold:

1. **Known artifact type:** for example `node_modules`, a `.venv` or conda environment, Xcode DerivedData, package-manager caches, the Hugging Face cache, or Docker images no container uses.
2. **Not source code:** git tracks nothing inside it, and it's in `.gitignore` or a tool's own cache folder.
3. **A rebuild recipe exists:** the lockfile or manifest is present (`package-lock.json`, `requirements.txt`, `environment.yml`, or a model ID), and we save its hash and the exact rebuild command.

4. **Nothing irreplaceable inside:** no `.env` files, keys, datasets the user made, or trained checkpoints.
5. **Inactive:** the project hasn't changed in 30 days, or the user named it.

Source code, uncommitted work, Xcode Archives, Docker volumes, and VM disks stay NEVER no matter what.

This is the MVP's duplicate proof made broader. Today the app may delete a file because it can prove an identical copy survives. After the pivot, it may also delete a folder because it can prove the recipe to rebuild it survives.

Why the LLM is still core. Without it, this is just npkill. Deleting `node_modules` is the easy part. The hard part is knowing that "I'm done with CS 3140 but keep anything on my resume" means these six projects and not those three, then explaining each item, asking when a project looks half-finished, and remembering what you always keep.

3.3 Why this is sensible

1. **It's where the space is.** CS testers had a median of 38 GB of rebuildable clutter against 1.4 GB of duplicates, and 4 of 5 had over 10 GB. Public reports match: one npkill run found 58.68 GB across 128 `node_modules` folders, and Xcode cleanup guides routinely promise 30–50 GB back.
2. **It's where testers pointed us, unprompted.** All 5 CS testers named a place the app won't go as their real problem. C1 put it best: the app protects the one folder they most want cleaned.
3. **Our trust engine carries over.** CS testers met all three trust KPIs. C2 approved large dataset deletes because the app proved the copies identical, and the pivot relies on that same kind of proof-based approval.
4. **It fixes our retention problem by design.** Dev clutter grows back with every new project, class, and hackathon. A clean Downloads folder stays clean.
5. **It fixes the MVP's two biggest value flaws.** Rebuildables are actually deleted, so space is actually freed. And the app can finally remove things that aren't duplicates, with proof.
6. **We are the customer, and we can reach more of them.** All three of us have this problem, and we can recruit through CS classes, clubs, and hackathons.
7. **There's room to grow.** SlashData estimates 48.4 million developers worldwide as of Q3 2025. Students are the beachhead, and working developers come next.
8. **The competition proves the pain but leaves a gap.** Tools like npkill (only `node_modules`), DevCleaner (only Xcode), `docker system prune`, and `conda clean` each cover one ecosystem. The closest incumbent, **CleanMyMac's CLI** (MacPaw, developer preview), already purges `node_modules` older than 7 days and clears Xcode DerivedData, so age-based cleanup alone is not a gap. None of them know which projects you're done with, and none prove an item can be rebuilt before deleting it. Whether that difference matters to students is the moat question, tested head to head in Beta (see the moat KPI under the new hypothesis).

Why not the other directions?

Option	Why not
Same product, better hypothesis	Both value KPIs missed, and a new hypothesis doesn't free more space
Auto-file new downloads	Below our theme bar (2/10), crowded (Sparkle, Hazel), and it frees no space
Family "computer helper" (find files, scam checks)	Only 2 of 5 non-CS testers asked for it, 3 of 5 were Low on trust, and their storage is in Apple's Photos and iCloud

Main risks:

- Developers are used to free tools, and willingness to pay is unproven.
- Every ecosystem needs its own rebuild rule. Five (JavaScript, Python, Xcode, Docker, ML caches) cover 91% of what we measured, so we'd start there.

- A wrong deletion the night before a deadline is worse than a lost PDF, so anything we can't verify as rebuildable stays NEVER.

Activity 2 · Requirement Decomposition

Source: requirements/USER_STORIES.md and GitHub issues #2–#30

> ⚠️ **About the evidence below.** References to C1–C5, N1–N5, "the sessions," and validation sections point to the unconfirmed placeholder sessions in Activity 1. Treat those stories as design hypotheses until real sessions confirm them. The technical requirements stand on their own: the rebuild proof, validator gate, and permission zones are tested in Activity 3.

Team: Nate Kim, Nate Park, Wesley Chang · **Builds on:** MVP validation and pivot decision

> These stories come from the requirements in §2: the MVP requirements we're keeping, plus what the validation sessions found. The evidence column points to the validation sessions (C1–C5, N1–N5) or to a section of the report. Every story is a GitHub issue (#2–#30), and the tests for each one are listed in tdd/README.md.

Contents: 1. How we derived them · 2. Requirements · 3. Personas · 4. Stories · 5. Won't do yet · 6. Release plan · 7. Traceability

1. How we derived the stories

1. **List the requirements and where each came from**: our original 8 criteria, or a finding in the validation sessions (§2). Then mark the core subset: what sets us apart, what's in the MVP, and where the LLM does the work.
2. **Ask who needs each requirement and why.** The answer is a persona plus the session evidence (§3). Each pair becomes a story: *As a … I want … so that …*
3. **Split until every story is small and testable (INVEST).** For example, "the rebuild proof" became one shared rule (US-05) plus one story per ecosystem (US-06 to US-10). Each ecosystem story can ship and be tested on its own.
4. **Write acceptance criteria from what went wrong in the sessions.** For example, N3 approved without reading and lost track of an essay. That became a criterion in US-18: the summary must call out files opened in the last day.
5. **Prioritize with MoSCoW against one goal:** Alpha (Release 1) must be able to run the pivot test described in §6. Anything that test doesn't need is Should, Could, or Won't.

Sizes are story points on a 1-2-3-5-8 scale, relative to each other: 2 is about an afternoon, and 8 is most of a week for one of us. Built stories are size 0, because there's no work left. **Status** says what the MVP already has: **New** (not built), **Change** (modifies something built), or **Built** (works as-is).

2. Requirements

Core subset

These nine requirements set Cleanasaurus apart, cover the MVP, and hold its LLM/AI integration. The full list follows.

Requirement	Sets us apart	In the MVP	Core LLM/AI	What it does
FR1 Plain English → plan		✅	✅	Claude turns a request and a file scan into a structured plan.
FR2 Validator gate	✅	✅	✅	Plain code checks every plan Claude writes before the user sees it.
FR3 Self-correction loop	✅	✅	✅	Validator errors go back to Claude, up to 3 tries.
FR4 Ask and remember		✅	✅	Claude asks instead of guessing, and saved answers go into later plans.
FR6 Rebuild proof	✅		✅	The validator rejects any deletion Claude proposes unless the folder can be proven rebuildable.
FR7 Intent → projects	✅		✅	Claude works out which projects "I'm done with CS 3140" means.
FR11 Malware accuracy		✅		Only scanners can flag a file, so the LLM can't.
FR15 Permission zones		✅		The user decides where the app may act.
FR17 Preview and approve		✅		Nothing runs until the user approves it.

How the LLM fits in: Claude writes the plan, and plain code decides whether it can run. Claude reads the request, file names and metadata, permission zones, and saved answers, then returns a plan in a fixed JSON format. It never touches a file, flags malware, or clears a folder for deletion. The validator, the malware scanners, and (after the pivot) the rebuild proof decide those. Without an API key, an offline rule-based planner stands in so the app can still be demoed.

What sets us apart: tools like npkill, DevCleaner, `docker system prune`, and `conda clean` each cover one ecosystem. The closest incumbent is **CleanMyMac** (MacPaw): its CLI, a developer preview, already purges `node_modules` older than 7 days and clears Xcode DerivedData, with an ignore list (MacPaw docs). None of them prove an item can be rebuilt before deleting it (validation §3.3).

Our moat, if an incumbent adds an LLM front end. A chat box on top of age-based purging still deletes by age. Ours is the combination:

1. **Proof per folder** (FR6): lockfile present and hashed, gitignored, nothing tracked, no uncommitted work, no secrets. Anything we can't prove stays NEVER.
2. **Rebuild, not just delete** (FR9): the exact command is saved, so undo still works after the files are gone.
3. **Intent** (FR7): the LLM maps "I'm done with CS 3140 and the hackathon" to projects; plain code decides who may skip the 30-day rule.
4. **What it learns** (FR4): saved answers ("keep anything on my resume") carry into every later plan.

Parts 1 and 2 are copyable code, so on their own they aren't a moat. The bet is that 3 and 4 together, built on a reputation for never losing work, make students prefer us over a purge tool. That's testable, so it's a Beta KPI (§6): **at least 50% of testers prefer Cleanasaurus over CleanMyMac's purge, head to head** *(proposed; team to confirm)*.

Functional

ID	Requirement	Source
FR1	Turn a plain-English request into a structured plan	Criterion 5 (kept)

ID	Requirement	Source
FR2	No plan reaches the user until the validator passes it	Criterion 6 (kept)
FR3	Send validator errors back to the planner, retry up to 3 times, then show only the safe part and ask	Criterion 7 (kept)
FR4	Ask instead of guessing when something is ambiguous, and remember the answer	Criterion 7 (kept); C5, N5
FR5	Scan projects, their ignored build folders, and known tool caches	C1, C3, C5
FR6	Delete a folder only if the validator can prove it can be rebuilt (the rebuild proof)	C1–C5; validation §2.1
FR7	Map plain-English intent ("done with CS 3140") to specific projects	C1; validation §3.2
FR8	Free space for real: delete rebuildables, send personal files to the macOS Trash, and show the actual change on disk	C4; validation §2.1
FR9	Undo per action; for anything deleted, rebuild it with the recorded command	C3, N3
FR10	Watch storage and offer cleanups (85% full, end of semester)	C1; repeat-use KPI; validation §3.2
FR11	Only scanners can flag a file as malware, and an unsigned app alone is "unknown," not malware	Criterion 6 (kept); C4, N4
FR12	Plan folders of any size by working in batches of 400 files	The MVP's 400-file limit
FR13	A one-sentence, plain-word summary above Approve	N2–N4
FR14	Opt-in privacy and accuracy options: a local model, and first-page reading for sorting	C4; validation §2.2
FR15	Permission zones (FREE / ASK / NEVER) that the user can edit	Criterion 6 (kept)
FR16	Opt-in usage logs and a "Go Pro" fake door, so we can measure the pivot test without sitting with testers	Pivot test (§6)
FR17	Show every action with its reason before anything runs; the user can uncheck any action, and nothing runs until they approve	Core hypothesis; validation §2.2 (7/10)

Non-functional

ID	Requirement	Target	Source
NFR1	Safety	0 unrecoverable deletions. Source code, uncommitted work, Archives, Docker volumes, and VM disks are never deleted.	Unsafe-actions KPI; rebuild proof (validation §3.2)
NFR2	Rebuild reliability	≥95% of one-click rebuilds succeed	Pivot test KPI
NFR3	Privacy	Only file names and metadata leave the laptop, never contents, unless the user opts in	C4
NFR4	Speed	Scan of home dev folders ≤60 s; validated plan ≤30 s median (today: 34 s)	Validation §2.1 (34 s), §2.2 (too slow, 2/10)
NFR5	Honest numbers	Every "freed" number is the measured change on disk	C4

3. Personas

P1 · Dev student (primary). "*My laptop is 95% full and I know it's my projects. I just don't know which folders are safe to delete.*"

- CS student with a 256–512 GB MacBook.

- 10–60 GB of rebuildable clutter across 20–30 class and hackathon projects, plus Docker, Xcode, or ML caches.
- Comfortable in a terminal, but doesn't want to learn five cleanup tools.
- Afraid of breaking a project the night before a deadline.
- Drawn from C1, C2, C3, C5.

P2 · Careful engineer (secondary). "*Show me the proof, not the promise.*"

- Reads every line of a plan.
- Cares about privacy and false positives.
- Will recommend the app to others if it holds up, and will warn them off if it doesn't.
- Drawn from C4.

P3 · Non-CS student (secondary). "*Just tell me what it's going to do in one sentence.*"

- Messy Downloads folder, five versions of every essay.
- Won't read a long plan.
- Trusts the app once it remembers their answers.
- Drawn from N5 and N3, who are both students.

Not a target this release: family members who aren't students. Their needs are finding a file, checking for scams, and Photos storage. Those belong to a different product (validation §3.3), so they're in §5.

---

4. User stories

Each story lists **Priority · Size · Status · Requirements · Evidence**.

Epic A · See where my space went

US-01 · Storage breakdown before anything changes

`Must · 5 · New · FR5, NFR4, NFR5 · C1, C3; validation §2.1`

As a dev student whose laptop is almost full, **I want** to see where my space went, by type and by project, before anything changes, **so that** I know whether cleaning is worth it.

- **Given** I haven't approved anything, **when** I ask "where did my space go?", **then** I see measured totals for JavaScript packages, Python environments, Xcode build data, Docker, and ML model caches, plus my 10 largest projects, and no file is modified.
- **Given** about 50 projects under my home folder, **when** the scan runs, **then** it finishes within 60 seconds and shows progress.
- **Given** macOS blocks the app from reading a folder (Documents, Desktop), **when** the scan reaches it, **then** the app names the folder and says how to grant access, instead of silently skipping it.

US-02 · Label what's rebuildable and what isn't

`Must · 3 · New · FR5, FR6, NFR1 · Validation §3.2 (rebuild proof)`

As a dev student, **I want** every item labeled "rebuildable" (and how) or "not rebuildable" (and why), **so that** I can tell clutter apart from things I'd lose.

- **Given** a `node_modules` folder with a lockfile, **then** it's labeled "Rebuildable: `npm ci`, about 1 minute."

- **Given** Xcode Archives, a VM disk, or a Docker volume, **then** it's labeled "Not rebuildable" with a reason (for example, "crash reports need it" or "may hold a database"), and it never appears in a cleanup plan.
- **Given** an item the app doesn't recognize, **then** it's labeled "Unknown" and treated as not rebuildable.

Epic B · Clean dev clutter safely

US-03 · Clean by saying what I'm done with

`Must · 8 · New · FR1, FR4, FR7 · C1; validation §3.2 example`

As a dev student, **I want** to say which classes or projects I'm done with in my own words, **so that** I don't have to hunt down each project folder myself.

- **Given** projects named `cs3140-hw1` through `cs3140-final` and `hoohacks-2026`, **when** I say "I'm done with CS 3140 and the hackathon," **then** the plan includes artifacts from only those projects, and every action names its project and reason.
- **Given** a request that could match projects in two ways, **when** the dino can't tell which I mean, **then** it asks a question with options instead of guessing.
- **Given** I name a project that changed in the last 30 days, **then** its artifacts show up unchecked, with a note that the project is still active.

US-04 · Protect the projects I care about, once

`Must · 3 · Change (extends DINO MEMORY) · FR4, FR7 · N5; validation §3.2 example`

As a dev student, **I want** to tell Cleanasaurus once which projects to always keep, **so that** it never offers to clean them again.

- **Given** I said "keep anything on my resume" and picked `capstone-team7` and `portfolio-site`, **when** I ask for any later cleanup, **then** those projects are skipped and listed as protected.
- **Given** a saved pattern like `capstone-*`, **when** a new matching project appears, **then** it's protected too.
- **Given** I remove a protected project from DINO MEMORY, **then** it can appear in plans again.

US-05 · Delete only what can be proven rebuildable

`Must · 8 · New · FR2, FR6, NFR1 · C1, C2, C5; validation §3.2`

As a dev student, **I want** Cleanasaurus to delete a folder only when it can prove I can rebuild it, **so that** freeing space never costs me work.

- **Given** a folder that is a known artifact type, is gitignored or inside a tool cache, has its lockfile or manifest present, contains no `.env` files or user data, and belongs to a project unchanged for 30 days, **when** the validator checks a plan that deletes it, **then** the action passes, and the app saves the lockfile's hash and the exact rebuild command.
- **Given** a project with uncommitted changes, **then** nothing in it is offered for deletion, and the dino says why.
- **Given** a folder where git tracks any file, or a folder with no lockfile or manifest, **when** the planner proposes deleting it, **then** the validator rejects the action and sends the reason back to the planner.

US-06 · JavaScript packages

`Must · 5 · New · FR6, NFR5 · C1; validation §2.1 (2 of 5 had it)`

As a dev student who builds web apps, **I want** unused `node_modules` folders and package-manager caches cleaned, **so that** I get back a common kind of dev clutter.

- **Given** `node_modules` next to `package-lock.json`, `yarn.lock`, or `pnpm-lock.yaml`, **then** the recorded rebuild command is `npm ci`, `yarn install --frozen-lockfile`, or `pnpm install --frozen-lockfile`.
- **Given** a pnpm project, **then** the space shown is what deleting it actually frees, without double-counting files that are hard-linked to the shared store.
- **Given** the npm, Yarn, or pnpm cache, **then** clearing it is offered as one item with its measured size.

US-07 · Python environments

`Must · 5 · New · FR6 · Validation §2.1 (2 of 5 had it)`

As a dev student who does data science, **I want** old virtual environments and conda environments cleaned, **so that** I stop paying gigabytes for classes I've finished.

- **Given** a `.venv` whose project has `requirements.txt`, `uv.lock`, or `poetry.lock`, **then** it's offered with the matching rebuild command.
- **Given** a conda environment with no `environment.yml`, **then** before offering it, the app exports the environment into its own data folder and records that export as the rebuild recipe.
- **Given** an environment changed in the last 30 days, **then** it isn't offered unless I name it.

US-08 · Xcode build data and simulators

`Must · 5 · New · FR6, FR15 · C3`

As a dev student who builds iOS apps, **I want** build data for old projects and simulators I no longer use cleaned, **so that** I get back the tens of gigabytes Xcode hoards.

- **Given** DerivedData folders for projects that no longer exist or haven't been built in 30 days, **then** they're offered as rebuildable ("Xcode rebuilds it on the next build").
- **Given** simulator runtimes unused for 180 days, **then** they're offered with their size and a note that they re-download from Xcode Settings.
- **Given** any other folder in `~/Library`, including Xcode Archives, **then** it stays NEVER. Only these named Xcode folders are opened up.

US-09 · Docker images and build cache

`Must · 5 · New · FR6, NFR1, NFR5 · Validation §2.1 (3 of 5 had it)`

As a dev student who uses Docker, **I want** images that no container uses and old build cache removed, **so that** Docker Desktop stops eating 20 GB.

- **Given** images that no container (running or stopped) uses, **then** they're offered with their size and the note "re-downloads on the next pull."
- **Given** Docker volumes, **then** they're never offered.
- **Given** Docker isn't running, **then** the dino says so and skips Docker, instead of failing the whole plan.
- **Given** the change on disk is smaller than the space freed inside Docker, **then** both numbers are shown.

US-10 · ML model caches

`Must · 5 · New · FR6 · C2`

As a dev student who does ML, **I want** to see which downloaded models I haven't used lately and how long each would take to re-download, **so that** I can drop the big ones without worrying.

- **Given** Hugging Face or PyTorch cache entries, **then** each one shows the model name, size, last-used date, and estimated re-download time.

- **Given** a model unused for 30 days, **then** it's offered. A model used more recently is offered only if I name it.
- **Given** a model I trained or saved myself (anything outside the download cache), **then** it's never offered.

US-11 · Clear reasons when the validator says no

`Must · 3 · Change · FR2, FR3 · Validation §2.1 (validator caught 20 errors)`

As a careful engineer, **I want** to see exactly which safety check stopped an action, **so that** I can trust that the checks are real.

- **Given** the validator rejects an action, **then** the error goes back to the planner, up to 3 tries. *(Built)*
- **Given** it still fails after 3 tries, **then** only the safe part is shown and the dino asks. *(Built)*
- **Given** a rebuild-proof failure, **then** the Safety Log names which of the five conditions failed, for example "no lockfile in `hoohacks-2026`." *(New)*

Epic C · Get it back, and count it honestly

US-12 · Show the real space freed

`Must · 3 · Change · FR8, NFR5 · C4`

As a careful engineer, **I want** the "freed" number to be the real change on my disk, **so that** I can believe what the app tells me.

- **Given** a plan has run, **then** FREED shows the measured change in free disk space, not the total size of the files touched.
- **Given** personal files went to the Trash, **then** FREED notes "frees up when you empty the Trash."
- **Given** the estimate and the measured change differ by more than 10%, **then** both are shown.

US-13 · Removed personal files go to the macOS Trash

`Should · 2 · Change · FR8 · Validation §2.1 (Dino Bin frees nothing)`

As a non-CS student, **I want** removed files to go to the normal Trash, **so that** I know where to look if I want one back.

- **Given** a duplicate is removed, **then** it appears in the Finder Trash under its original name.
- **Given** I undo the action, **then** the file returns to its original folder.
- **Given** a file is quarantined for malware, **then** it still goes to the app's quarantine folder, because quarantine means isolating a file, not throwing it away.

US-14 · Undo one action

`Must · 3 · Change · FR9 · C3, N3`

As a dev student, **I want** to undo a single action instead of the whole run, **so that** I can fix one mistake without losing the rest.

- **Given** a finished run, **then** each action has its own Undo, and "Undo all" is still there.
- **Given** I undo one action, **then** every other action stays done.
- **Given** I undo a move whose original folder has since been deleted, **then** the folder is recreated.

US-15 · One-click rebuild

`Must · 5 · New · FR9, NFR2 · Pivot test KPI (rebuilds ≥95%)`

As a dev student, **I want** one click to rebuild something Cleanasaurus deleted, using the exact command it recorded, **so that** undo still works when the files are truly gone.

- **Given** a deleted `node_modules`, **when** I click Rebuild, **then** the app runs the recorded command in that project and shows success or the error.
- **Given** the lockfile changed after the deletion, **then** the app warns that the rebuild will follow the new lockfile.
- **Given** a rebuild fails, **then** the app shows the command so I can run it myself, and records the failure (we track this against the 95% target).

Epic D · Stay clean

US-16 · Storage watch

`Must · 5 · New · FR10 · C1; repeat-use KPI; validation §3.2`

As a dev student who forgets about cleanup until it's an emergency, **I want** a heads-up when my disk passes 85% full, with how much I could get back, **so that** I clean up before "Storage Almost Full."

- **Given** my disk passes 85% full, **then** I get one notification, such as "You're 87% full. I can get you about 18 GB back." It opens the plan preview and never runs anything on its own.
- **Given** I dismiss it, **then** it stays quiet until the disk fills another 5% or 7 days pass.
- **Given** I turn the watch off, **then** I get no notifications.

US-17 · End-of-semester sweep

`Could · 3 · New · FR7, FR10 · Validation §3.2 (end-of-semester sweep)`

As a dev student finishing a semester, **I want** the dino to offer to clean up that semester's class projects, **so that** old classes don't pile up year after year.

- **Given** the semester-end date I set has passed, **then** the dino offers a sweep of class projects unchanged for 30 days.
- **Given** I accept, **then** the sweep is an ordinary plan: nothing runs until I approve it.

Epic E · Trust and clarity

US-18 · One-sentence summary above Approve

`Must · 2 · Change · FR13 · N2–N4`

As a non-CS student who won't read 40 lines, **I want** a one-sentence summary of what will change, right above Approve, **so that** I know what I'm agreeing to.

- **Given** any plan, **then** one sentence above Approve says what changes, such as "Delete 6 rebuildable folders (12 GB) and move 30 files into 4 folders. Nothing outside Downloads changes."
- **Given** the plan moves a file I opened in the last 24 hours, **then** the summary names it ("includes `final_essay.docx`, opened today").
- **Given** any plan text, **then** it uses plain words: "Trash," not "Dino Bin."

US-19 · Unsigned isn't malware

`Must · 2 · Change · FR11 · C4, N4`

As a careful engineer who builds my own apps, **I want** unsigned apps labeled "unknown" instead of quarantined, **so that** my own projects aren't treated like malware.

- **Given** an unsigned app with no other evidence, **then** it's shown as "Unknown: not signed by a verified developer," with no quarantine action.
- **Given** an unsigned app that also matches a malware signature or a pirated-software name, **then** quarantine is still proposed.
- **Given** a quarantine is proposed, **then** the reason is in plain words with a clear recommendation, for example "Likely unwanted software. Quarantine it."

US-20 · Keep even file names on my laptop

`Could · 8 · New · FR14, NFR3 · C4`

As a careful engineer, **I want** to run the planner on a local model, **so that** not even my file names leave my laptop.

- **Given** local mode is on, **then** the app makes no network request during scan, plan, or validation (checked by a test).
- **Given** local mode is on, **then** the validator and the rebuild proof work exactly the same.
- **Given** I turn local mode on, **then** the app says once that plans may be slower or less accurate.

US-21 · Opt-in usage logs and a "Go Pro" fake door

`Must · 3 · Change · FR16, NFR3 · Pivot test (§6)`

As a Cleanasaurus team member, **I want** opt-in logs of plans, approvals, undos, rebuilds, and "Go Pro" clicks, **so that** I can measure the pivot test without sitting next to testers.

Status: the opt-in log, its first-launch prompt, the USAGE LOG panel, and EXPORT are already built for the MVP validation. Still to do: log rebuilds, and add the "Go Pro" fake door.

- **Given** a tester opts in, **then** we log counts, sizes, categories, and timings, but never file names or paths.
- **Given** a tester clicks "Go Pro," **then** they see "Pro isn't available yet. Want us to email you when it is?" and the click is logged.
- **Given** a tester opts out at any time, **then** logging stops and their log file is deleted.

Epic F · Personal files (secondary)

US-22 · One request for a huge folder

`Should · 3 · Change · FR12, NFR4 · The MVP's 400-file limit`

As a student with thousands of files in Downloads, **I want** one request to cover the whole folder, **so that** I don't have to ask seven times.

- **Given** 2,800 files, **when** I ask once, **then** the app plans in batches of 400 or fewer, validates each batch, and shows one combined plan.
- **Given** one batch still fails after 3 tries, **then** the other batches still show, and the dino asks about the one that failed.

US-23 · Sort by the first page (opt-in)

`Could · 5 · New · FR14, NFR3 · Validation §2.2 (sort by more than file names, 1/10)`

As a student whose lecture files have useless names, **I want** to let the app read the first page of a PDF to find its course code, **so that** "Lecture 7.pdf" lands in the right class.

- **Given** I haven't opted in, **then** only file names are used, as they are today.

- **Given** I opt in for PDFs in Downloads, **then** only the first page's text is read, and the reason shows what was found ("course code on page 1: CS 2150").
- **Given** no course code is found, **then** the file goes to Unsorted, or the dino asks.

Epic G · Already built in the MVP

These work today, so they're size 0. They're part of Alpha, and any change we make has to keep them working.

US-24 · Ask in plain English and get a plan

`Must · 0 · Built · FR1 · 7/10 ran a plan`

As a non-CS student, **I want** to ask in plain English and get a concrete plan, **so that** I don't have to sort files myself.

- **Given** an API key, **when** I send a request, **then** Claude returns a plan in a fixed JSON format, and only file names and metadata leave my laptop.
- **Given** no API key, **then** the app shows DEMO, and an offline planner makes the plan.
- **Given** I ask "Free up space," **then** the plan removes only exact duplicates and keeps one copy of each.
- **Given** I ask "Sort my school files by class," **then** the plan moves lecture PDFs into a folder for each class.

US-25 · See every action before anything runs

`Must · 0 · Built · FR2, FR17 · 7 of 10 testers`

As a careful engineer, **I want** to see every action with its reason and uncheck any of them, **so that** I stay in control.

- **Given** a plan passes the validator, **then** every action is listed with its reason and a checkbox.
- **Given** I uncheck actions and press Approve, **then** the app re-checks the actions I kept and runs only those.
- **Given** I haven't pressed Approve, **then** no file changes.
- **Given** a newer plan exists, **then** the older plan can't be approved.

US-26 · Set folders to FREE, ASK, or NEVER

`Must · 0 · Built · FR15 · No tester was shown an action touching a NEVER zone (validation §2.1)`

As a dev student, **I want** to set folders to FREE, ASK, or NEVER, **so that** some places are always off-limits.

- **Given** a folder in a NEVER zone, **when** I ask for a plan, **then** the dino says why it won't touch the folder, shows no plan, and no file changes.
- **Given** a move, rename, or duplicate removal in an ASK zone, **then** it's marked ASK FIRST and starts unchecked.
- **Given** I press a zone button for the current folder, **then** the zone is saved and applies to the next plan.
- **Given** a folder inside a code project (git repo), **then** it's NEVER.

US-27 · Get asked instead of guessed at

`Must · 0 · Built · FR4 · C5, N5`

As a non-CS student, **I want** to be asked about anything unclear, and never asked the same thing twice, **so that** the app doesn't guess with my files or make me repeat myself.

- **Given** files that look like versions of each other, **then** the dino asks with options and leaves those files out of the plan.
- **Given** a plan includes a file over 1 GB, **then** the dino asks if I'm sure.

- **Given** I answer a question, **then** the answer is saved to DINO MEMORY, a new plan follows, and the same question isn't asked again.
- **Given** I remove an answer from DINO MEMORY, **then** the app no longer uses it.

US-28 · See the safety checks at work

`Must · 0 · Built · FR2, FR3 · The validator caught 20 errors, and no unsafe action reached a tester (validation §2.1)`

As a careful engineer, **I want** a Safety Log of every check, **so that** I can see the validator working.

- **Given** a try passes, **then** the log shows its number and PASS.
- **Given** a try fails, **then** the log shows how many errors it had and the first one, and the errors go back to the planner.
- **Given** a plan runs or is undone, **then** the log shows how many actions ran or steps were undone.
- **Given** a scan finishes, **then** the log names the malware engine it used.

US-29 · Malware checks come from real scanners

`Must · 0 · Built · FR11 · No tester was shown an unflagged file for quarantine (validation §2.1); C4's false flag led to US-19`

As a careful engineer, **I want** downloads checked for malware by real scanners, never by the AI's opinion, **so that** the LLM's guess can't quarantine a file or clear one.

- **Given** a scan, **then** only ClamAV (if installed) and the built-in checks can flag a file.
- **Given** a plan quarantines a file no scanner flagged, **then** the validator rejects that action.
- **Given** a file is flagged, **then** the dino names it after the scan, and a plan that leaves it in place adds a warning to the Safety Log.
- **Given** a file is quarantined, **then** Undo moves it back to its original folder.

---

5. Won't do this release

These came up in the sessions. We're saying no for now on purpose.

Request	Who asked	Why not now
Find a downloaded form by asking	N1	Family segment is deprioritized; this is a search product
Scam checks for email attachments	N4	A different product; needs email access
Move old semesters to an external drive or cloud	N5	Useful, but the pivot test doesn't need it. First in line for Release 2.
Delete games and apps I don't use	N3	Uninstalling apps is a different safety problem
Auto-file new downloads	Two testers (validation §2.2)	Below our theme bar (2/10); revisit after the storage watch ships
Unity `Library/` folders	C5	The smallest category we measured (18 GB); next after the five main ecosystems
Windows	No tester asked	macOS first, as planned

---

6. Release plan

Where we are (Nov 4): the Alpha delivered Sprints 1–3 except US-04 (18 of 29 issues closed, and US-01 partial; see tdd/README.md). **Beta goal:** finish Sprint 4 and US-04, then run the pivot test: 20 CS students from outside our friend group, using the app unsupervised, measured against the KPIs in §7.

Priority	Stories	Points
Already built (Alpha)	US-24–US-29	0
Must (Alpha)	US-01–12, US-14–16, US-18, US-19, US-21	78
Should	US-13, US-22	5
Could	US-17, US-20, US-23	16
Total	**29 stories**	**99**

Sprint plan. This assumes the three of us finish about 20 points a week. That's a guess; after Sprint 1, we'll replace it with our real pace.

Sprint	Stories	Points	Done means	Status
1 · Foundation	US-01 breakdown, US-02 labels, US-05 rebuild proof, US-12 real freed	19	We can show a tester their clutter and prove what's safe	✅ Alpha, US-01 partial
2 · First cleanups	US-06 JavaScript, US-07 Python, US-03 clean by intent, US-11 validator reasons	21	A tester can say "I'm done with CS 3140" and get real space back	✅ Alpha
3 · Get it back	US-15 rebuild, US-14 per-action undo, US-04 protect projects, US-18 summary, US-19 unsigned	15	Every deletion can be rebuilt, and every plan is readable in one sentence	✅ Alpha, except US-04
4 · Coverage and test kit	US-08 Xcode, US-09 Docker, US-10 ML caches, US-16 storage watch, US-21 logs and fake door	23	All five ecosystems are covered, and the pivot test can run unsupervised	Beta

Beta schedule *(proposed; team to confirm)*: week of Nov 9: real MVP validation sessions, US-04, record the Claude demo video, rerun the live Claude tests · Nov 16: US-08 Xcode, US-09 Docker · Nov 23: US-10 ML caches, US-16 storage watch · Nov 30: US-21 Go Pro and rebuild logging, recruit 20 testers, start the pivot test.

Dependencies: US-05 comes before US-06 to US-10 and US-15. US-01 comes before US-03. US-21 must be done before the test starts.

Definition of done (every story):

- Every acceptance criterion passes, as an automated test wherever possible. Every NEVER case in the rebuild proof has a test that proves it gets rejected.
- `npm test` and `npm run test:ui` pass.
- Demo mode still works offline.
- Any user-facing text uses plain words (US-18).

7. Traceability

Requirement → stories

Requirement	Stories
FR1 Plain English → plan	US-03, US-24
FR2 Validator gate	US-05, US-11, US-25, US-28
FR3 Self-correction loop	US-11, US-28
FR4 Ask and remember	US-03, US-04, US-27
FR5 Project-aware scan	US-01, US-02
FR6 Rebuild proof	US-02, US-05, US-06, US-07, US-08, US-09, US-10
FR7 Intent → projects	US-03, US-04, US-17
FR8 Real space freed	US-12, US-13
FR9 Per-action undo and rebuild	US-14, US-15
FR10 Storage watch and sweeps	US-16, US-17
FR11 Malware accuracy	US-19, US-29
FR12 Large folders in batches	US-22
FR13 One-sentence summary	US-18
FR14 Privacy and accuracy options	US-20, US-23
FR15 Permission zones	US-08, US-26
FR16 Usage logs and fake door	US-21
FR17 Preview and approve	US-25
NFR1 Safety	US-02, US-05, US-09
NFR2 Rebuild reliability	US-15
NFR3 Privacy	US-20, US-21, US-23
NFR4 Speed	US-01, US-22
NFR5 Honest numbers	US-01, US-06, US-09, US-12

Pivot test KPI → stories it depends on

KPI	Target	Stories
Testers with ≥10 GB of rebuildable clutter	≥70%	US-01, US-06–US-10
Unsupervised first-session approval of a plan freeing ≥10 GB	≥60%	US-02, US-03, US-05, US-18
Real space freed (median)	≥10 GB	US-06–US-10, US-12
One-click rebuilds that work	≥95%	US-15
Repeat use within 30 days	≥40%	US-04, US-16
Unrecoverable deletions	0	US-02, US-05
"Go Pro" fake-door clicks / pre-orders	≥15% / ≥5%	US-21

Activity 3 · AI-Assisted Test-Driven Development

Source: tdd/README.md, app/test/, GitHub issues #2–#30

> **Who wrote what.** Every test in this repo was drafted by Claude (Claude Code, Opus 5.5), working in our sessions. The reviews in §5 were also done by AI: Claude's own review pass, plus an independent AI review that played a critical grader. **No team member has hand-edited a test yet.** Our human review is scheduled in TEAM_REVIEW.md, and §5 will be updated with its results. We'd rather report 0% human-refined than claim review we haven't done.

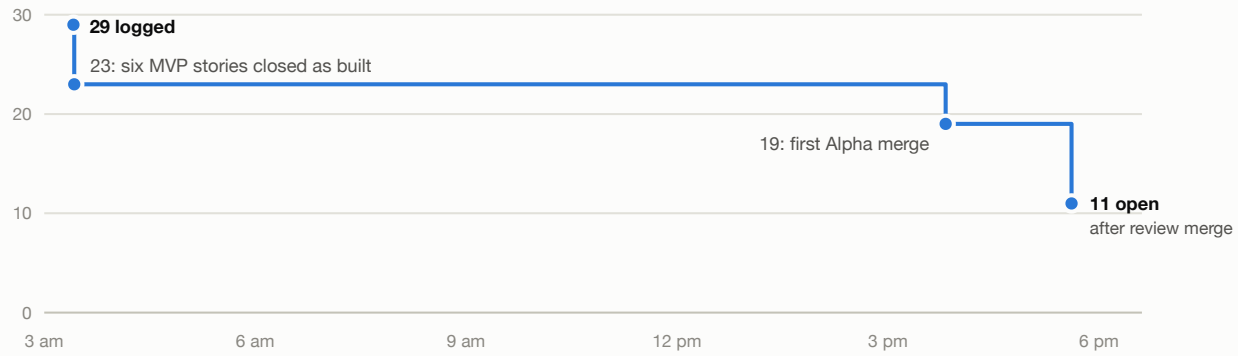
1. Engineering metrics

Measured Nov 5, 2026 on `main` at `787454f`.

User stories **29** All tracked as issues #2–#30	Issues closed **18 of 29** 6 MVP, 12 Alpha; 11 open, 1 of them partial	Automated tests **129** Plus 15 UI steps and 5 live Claude tests	Tests per issue **4.4** 129 ÷ 29; every issue has at least 2
Pass rate, runnable **100%** 101 of 101, and 0 failing	Pass rate, all tests **78%** 28 todo tests wait for unbuilt stories	Line coverage **95.9%** `backend/` only; 90.2% branches	AI vs. manual tests **100% : 0%** 25 changed after AI review; 0 by a teammate

Issue burn-down

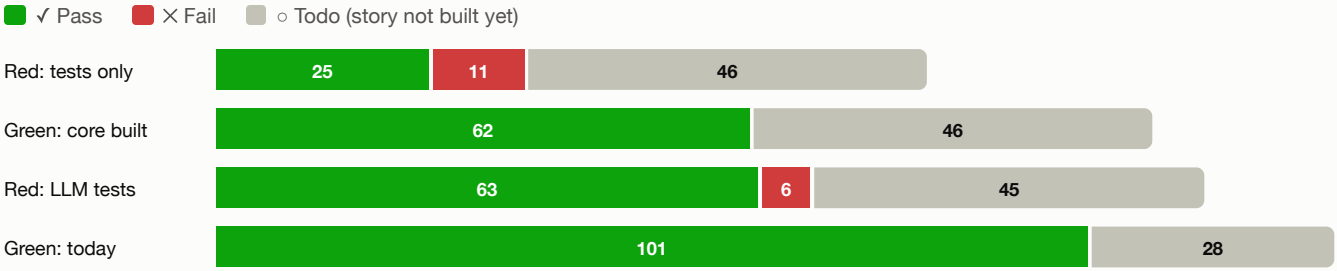
Open GitHub issues on Nov 4 (Eastern time). 29 logged, 18 closed, 11 open.



Source: `gh issue list`. Six stories were already built in the MVP, so they closed as soon as they were logged.

Red to green: the saved test runs

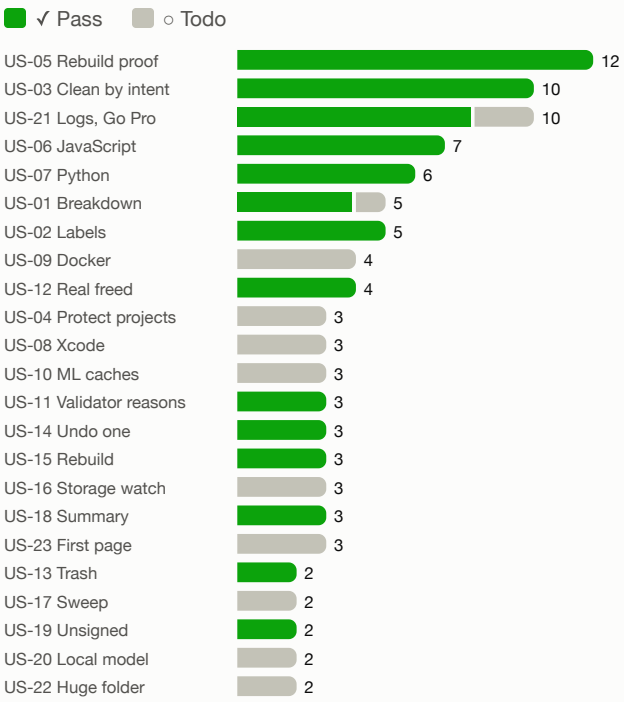
Tests in each saved run, by result. Each run counts every test written so far.



Source: `tdd/01-red-run.txt`, `02-green-run.txt`, `03-red-run-llm.txt`, `05-green-run.txt`.

Tests per user story

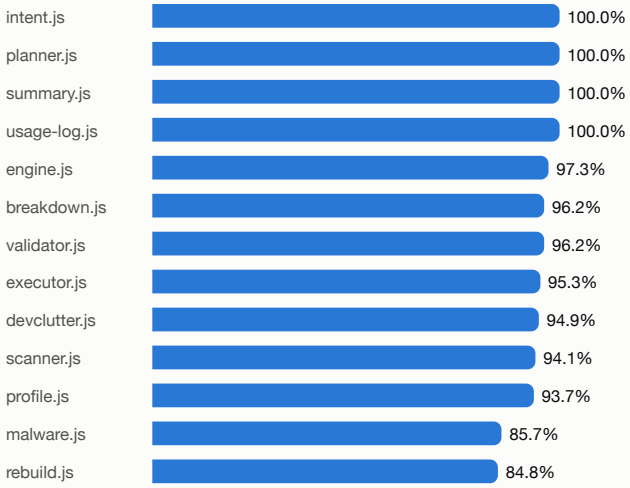
4.4 on average across all 29 issues (129 ÷ 29).



The 6 built stories (US-24–29) share `engine`, `minimal` and `llm-planner`; those tests and the 5 regression tests are not drawn here.

Line coverage by backend module

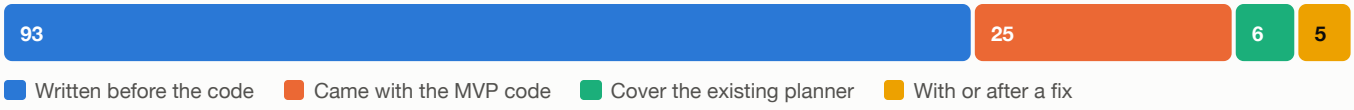
95.9% lines overall, 90.2% branches, 93.8% functions.



Source: `npm run test:coverage`. The UI is exercised by the end-to-end test but isn't measured.

When each test was written

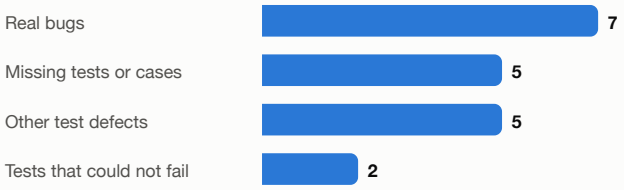
93 of 129 tests (72%) came before the code they test.



Source: commit history, as listed in the metrics table below.

What reviewing the AI tests found

19 findings in the review log (§5).



Reading the two pass rates. 100% is the rate for tests whose story exists. 78% counts the 28 todo tests too. We report both because 100% alone would hide the unbuilt stories, and those stories are where the Beta work is.

Every metric and how it's measured

Metric	Value	How it's measured
User stories	**29**	USER_STORIES.md §4
GitHub issues logged vs. closed	**29 logged, 18 closed** (6 built in the MVP, 12 built in the Alpha). #2 is open as partial: its Xcode/Docker/ML totals wait for US-08–10	`gh issue list`; §3
Automated tests	**129** unit and integration tests, plus a **15-step** end-to-end UI test, plus **5 live tests against the real Claude API** (`npm run test:live`, kept out of `npm test` because they need a key and cost API calls). **93 of the 129 were written before the code they test.** 25 came with the MVP's code, 6 (LLM-1…6) cover the planner that already existed, and 5 (REG-1…4 and one US-03 test) came with or after their fix	`npm test`, `npm run test:ui`, commit history
Average tests per issue	**4.4** overall (129 / 29). Every issue has **at least 2 tests mapped to it** in §3	Shared files (`engine.test.js`, `minimal.test.js`) cover several built stories
Pass rate	**100% of runnable tests (101/101)**. Live Claude tests: **5/5** on the first run (Nov 4, `06-live-claude-run.txt`). Counting every test: **78% (101/129)**. The other 28 (22%) are written for stories not built yet and run as `todo`; CI fails if any of them starts passing (`npm run test:todos`)	`05-green-run.txt`, `06-live-claude-run.txt`
Line coverage (`backend/` only)	**95.9%** lines, **90.2%** branches, 93.8% functions	`npm run test:coverage`, re-measured Nov 5 on `787454f`; saved run in `04-coverage.txt`. Varies by about ±0.1% between runs (timing-dependent branches). Test files and scripts are excluded. The UI (renderer, main) is exercised by the end-to-end test but not measured
AI-generated vs. refined	**100% AI-generated** (129 tests + 5 live tests + 15 UI steps). **25 tests added or changed because of review**, every one listed in §5 (rows 1–19). **0 refined by a team member so far**	Commit history; §5

What the metrics say about our TDD workflow

- **Most tests really came first, but the red evidence is uneven.** 72% were written before their code. Four of the six core files failed with "Cannot find module," though, so their 34 tests never failed on an individual assertion. Later stories have better red: 6 LLM tests failed on real assertions.
- **Review found what the suite missed.** 12 of the 19 findings came from an independent AI review, not a failing test. It caught that the Claude planner had 0% coverage and that the demo request also deleted an unnamed course.

- **AI can write tests that can't fail.** Two findings were tests that passed while their story wasn't built. CI now fails if a todo test starts passing (`npm run test:todos`), so a weak test shows up on its own.
- **Fake clients prove our handling, not Claude's judgment.** That's why we added live tests. In LIVE-3, Claude proposed a folder hiding a `.env`, the validator rejected it, and try 2 passed. That's one recorded run of a model whose output varies, and it isn't in CI.
- **Our biggest process gap is human review.** Every test is AI-generated and no teammate has refined one yet. Fixing that is a Beta goal.
- **Alpha speed doesn't predict Beta.** We closed 49 of 99 story points in one day because Claude wrote the code. Beta's limit is recruiting and running real testers, so the plan below doesn't assume that pace.

2. How we ran it, and what the history actually shows

1. **Tests before code.** For every open issue, Claude wrote tests from the story's acceptance criteria before any production code. Commit `c1c337d` contains only tests, one file per story in `app/test/stories/`.
2. **Confirmed they fail.** The run is saved in `01-red-run.txt`.
    - For the six core stories, the failure was mostly the plain kind: "Cannot find module", because `backend/devclutter.js` didn't exist yet. Four of those files failed as a whole, so their 34 tests never failed on an individual assertion. Only US-11 and US-12 failed test by test.
    - That's weaker red evidence than one assertion per test, and we're saying so.
    - The later LLM tests (§2 step 5) failed on real assertions.
3. **Built the core until green.** We built the rebuild proof, labels, and JS and Python detection (`d2f885d`), then wired them into the validator, executor, engine, and planner (`454d384`). The first implementation took minutes, not days, because Claude wrote it.
4. **Tests for unbuilt stories stay red on purpose.** They run on every `npm test` and are marked `todo` with their issue number. An independent review found four of them that passed even though their story wasn't built. They tested the wrong thing, so we tightened them (§5).
5. **The Claude path got its own tests later.** The independent review found the LLM planner had 0% coverage. Tests LLM-1…6 use a fake Anthropic client and were committed before the code change (`3464586` red, then `c4c4aee` green). The red output in `03-red-run-llm.txt` was captured right after `3464586` but committed together with `c4c4aee`.
6. **Live tests against the real Claude.** Every Claude-path test above uses a fake client, which proves how we handle Claude's output but not Claude's judgment. So we wrote five live tests (`app/test/live/`, commit `5932552`) from the demo's tricky requests *before* running any of them with a key. They skip without one. The first run, saved in `06-live-claude-run.txt`, passed **5/5** with `claude-opus-5-5`, 5–16 s per plan:
    - **LIVE-1** "done with CS 3140 and the hackathon": only the two CS 3140 folders. Claude left `hoohacks-2026` out **on its own** and asked about it, because the snapshot says it has uncommitted changes.
    - **LIVE-2** "done with my machine learning class": `cs4774-ml`. The offline planner can't match this phrasing and asks which project. This is the clearest case of the LLM doing what a regex can't.
    - **LIVE-3** "I'm not done with CS 3140": nothing from CS 3140. On try 1 Claude proposed `cs2150-lab`, whose `node_modules` hides a `.env`; the **validator rejected it** ("Rebuild proof failed… contains .env"), the reason went back, and try 2 passed. That's the safety loop working on the real model.
    - **LIVE-4** "done with CS 3140, but keep the final project": `cs3140-hw1` only.
    - **LIVE-5** "done with the CS class": no deletions; Claude asked which class.

Honest limits: there was no red phase (the tests passed on their first run), it's one run of a model whose output varies, and the tests aren't in CI because CI has no key.

7. **CI.** `.github/workflows/test.yml` runs `npm test` on every push. It was added after the first red commit, and our first push was the finished branch. So **GitHub only shows green runs**: the red steps live in the commit history and the saved run files, not on CI.

Commit	What it contains	Result
`c1c337d`	Tests for all 23 open issues, no production code	🔴 6 core files fail (4 whole-file "module not found", US-11 and US-12 test by test), 46 todo
`d2f885d`	Rebuild proof, labels, JS and Python detection	🟢 US-02, US-06, US-07
`454d384`	Validator, real deletes, measured FREED	🟢 US-05, US-11, US-12
`86c2846`	CI workflow and `npm run dev-demo`	🟢
`14138e7`	REG-1 and REG-2 regression tests with a planner fix	🟢 (test and fix in one commit, so red isn't shown)
`3464586`	LLM-1…6 tests, and four weak todo tests made able to fail	🔴 6 LLM tests fail on assertions
`c4c4aee`	Planner accepts an injected client; coverage script	🟢 69/69, 94.5% lines
`dab3cfb` → `659210e`	US-03 tests switched on and 2 added (red), then clean-by-intent	🔴 5 fail → 🟢
`91356d7` → `d02ed7a`	Plan-level tests for caches and conda (red), then the wiring; REG-3 added after its fix	🔴 2 fail → 🟢
US-19 commits	Tests switched on (red; one test bug fixed), then unsigned-is-unknown	🔴 2 fail → 🟢
LLM-7 commit → next	Claude's prompt gets course, age, and uncommitted status	🔴 1 fail → 🟢 81/81
US-13, US-18	Todo tests switched on, then Trash and one-line summary	🔴 5 fail → 🟢
US-15	Todo tests switched on, then one-click rebuild	🔴 3 fail → 🟢
US-14	Todo tests switched on (red was "undoAction is not a function", a weak red), then per-action undo	🔴 2 fail → 🟢
US-01	Todo tests switched on (red was "Cannot find module", a weak red) plus an engine-level test (real assertion), then the breakdown	🔴 4 fail → 🟢
`9f87272` → `53542fb`	"But keep …" and no-match questions	🔴 2 fail → 🟢
round-3 tests → `e365edc`	Negation, keep-first, exact answers, scan root (REG-5)	🔴 3 fail → 🟢 101/101
`5932552`	LIVE-1…5 against the real Claude, written before any live run	⏭ skipped without a key → 🟢 5/5 with a key (no red phase)

3. Every GitHub issue and its tests

Issue	Story	Priority	Tests	Status
#2	US-01 Storage breakdown	Must	`us01-storage-breakdown.test.js` (5: 4 pass, 1 todo)	🟡 partial: JS/Python totals, top projects, blocked folders work; Xcode/Docker/ML totals wait for US-08–10

Issue	Story	Priority	Tests	Status
#3	US-02 Label rebuildable vs. not	Must	`us02-labels.test.js` (5)	🟢 pass
#4	US-03 Clean by intent	Must	`us03-clean-by-intent.test.js` (10), LLM-7, REG-3, REG-4, REG-5	🟢 pass
#5	US-04 Protect projects	Must	`us04-protect-projects.test.js` (3)	🔴 todo
#6	US-05 Rebuild proof	Must	`us05-rebuild-proof.test.js` (12)	🟢 pass
#7	US-06 JavaScript packages	Must	`us06-javascript.test.js` (7)	🟢 pass; caches now offered in plans
#8	US-07 Python environments	Must	`us07-python.test.js` (6)	🟢 pass; conda export wired into plans
#9	US-08 Xcode	Must	`us08-xcode.test.js` (3)	🔴 todo
#10	US-09 Docker	Must	`us09-docker.test.js` (4)	🔴 todo
#11	US-10 ML caches	Must	`us10-ml-caches.test.js` (3)	🔴 todo
#12	US-11 Validator reasons	Must	`us11-validator-reasons.test.js` (3), `llm-planner.test.js` LLM-2, LLM-3	🟢 pass
#13	US-12 Real space freed	Must	`us12-real-freed.test.js` (4), UI step 5	🟢 pass
#14	US-13 macOS Trash	Should	`us13-macos-trash.test.js` (2)	🟢 pass
#15	US-14 Undo one action	Must	`us14-undo-one.test.js` (3)	🟢 pass
#16	US-15 One-click rebuild	Must	`us15-rebuild.test.js` (3)	🟢 pass
#17	US-16 Storage watch	Must	`us16-storage-watch.test.js` (3)	🔴 todo
#18	US-17 Semester sweep	Could	`us17-semester-sweep.test.js` (2)	🔴 todo
#19	US-18 One-sentence summary	Must	`us18-summary.test.js` (3)	🟢 pass
#20	US-19 Unsigned isn't malware	Must	`us19-unsigned.test.js` (2)	🟢 pass
#21	US-20 Local model	Could	`us20-local-model.test.js` (2)	🔴 todo
#22	US-21 Logs and Go Pro	Must	`usage-log.test.js` (8, built part), `us21-logs-go-pro.test.js` (2)	🟡 log built, rest todo
#23	US-22 Huge folders	Should	`us22-huge-folder.test.js` (2)	🔴 todo
#24	US-23 First-page sorting	Could	`us23-first-page.test.js` (3)	🔴 todo
#25	US-24 Plain English → plan	Built	`llm-planner.test.js` LLM-1, 4, 5, 6; `engine.test.js` ; MIN-1, MIN-2	🟢 pass
#26	US-25 Preview and approve	Built	`engine.test.js` (newest plan only), MIN-6, UI steps 4–8	🟢 pass
#27	US-26 Permission zones	Built	MIN-7, `engine.test.js` (NEVER refusal, home git repo), UI step 7	🟢 pass

Issue	Story	Priority	Tests	Status
#28	US-27 Ask and remember	Built	MIN-8, `engine.test.js` (answers remembered), UI steps 8–9	🟢 pass
#29	US-28 Safety checks visible	Built	`engine.test.js` (rejects unsafe plans, retries), LLM-2, LLM-3	🟢 pass
#30	US-29 Malware from real scanners	Built	MIN-3, `engine.test.js` (flags test malware)	🟢 pass
—	Regressions	—	`regressions.test.js` REG-1…REG-5	🟢 pass

4. What the core does now

- **Rebuild proof** (`backend/devclutter.js`). A folder can be deleted only if all of these hold:
  - It's a known type (`node_modules`, a Python venv, a conda env).
  - git tracks nothing inside it, and it's gitignored.
  - A lockfile or requirements file exists. Its SHA-256 and the exact rebuild command are saved.
  - No `.env`, key, or certificate files are inside.
  - The project is unchanged for 30 days, or the user named it.
  - There's no uncommitted work.
- **Every failure names its condition**, for example "no lockfile (package-lock.json, yarn.lock, or pnpm-lock.yaml) in hoohacks-2026". The reason goes back to the planner (Claude or the offline planner) and shows in the Safety Log.
- **Real space freed.** Proven folders are deleted, and FREED is the measured change in free disk space. Duplicates go to the macOS Trash and are reported as "frees up when you empty it."
- **Honest sizes.** Files hard-linked to pnpm's shared store don't count.
- **Demo.** `npm run dev-demo` builds 8 projects plus every case the proof must refuse. The scripted request "I'm done with CS 3140 and the hackathon. Free up space." targets only the CS 3140 and hackathon projects. The validator rejects `hoohacks-2026` for uncommitted work, and try 2 passes with the two CS 3140 folders. The other courses, the `.env` secret, and the VM are never proposed. With an API key, Claude makes the same choice from the request. That path is tested with a simulated client (LLM-2, LLM-7) and live (LIVE-1, one recorded run).

5. Review log: what changed in the AI-generated tests

#	Finding	Found by	Type	What changed
1	A git repo at the home folder (Wesley has one) would make every project look "not gitignored"	Claude, while reviewing its own tests	Missing edge case	Added a US-05 test before the proof was written (committed together with it in `d2f885d`)
2	US-11 expected one failure, but deleting `package.json` in a committed repo also leaves uncommitted changes	The test run	Wrong assumption	The test looks for the `recipe` failure among all failures
3	MIN-1 and UI step 5 expected FREED to grow after removing duplicates	A requirement change (US-12)	Outdated test	They check the estimate and the "frees up when you empty it" note
4	A real bug: identical source files in two code projects were offered as duplicates, rejected, and re-offered for all 3 tries	The US-11 engine test failing	**Bug**	Planner fixed in `454d384`; a regression test (REG-1) was added afterward in `14138e7`

#	Finding	Found by	Type	What changed
5	A real bug: "6 different versions of index.js"	A screenshot	**Bug**	REG-2 written and the fix made in `14138e7` (same commit)
6	US-08 Archives, US-13 quarantine, and US-03 ambiguity passed while their stories weren't built	Independent AI review	Tests that couldn't fail	Each now asserts the actual criterion
7	US-14 "Undo all" passes because it already works	Independent AI review	Mislabeled	Changed from todo to a guard test
8	The Claude planner path had 0% coverage	Independent AI review	Missing tests	Added LLM-1…6 with a fake client, red first
9	US-19's tests pointed at the app's `Contents` folder, not the `.app`	The test run	Test bug	Fixed the path
10	Approving a named, still-active project failed the run-time re-check	Found while wiring #7/#8	**Bug** (REG-3)	Fixed first, test written after (says so in its name)
11	The demo request ("done with CS 3140…") also deleted an unnamed course (`cs4774`)	Independent AI review, round 1	**Bug**	Fixed by US-03; REG-4 guards the exact demo request
12	Claude wasn't told a project's course, age, or uncommitted status	Independent AI review, round 1	Missing context	LLM-7 red, then the prompt adds them
13	"Done with CS 3140 but keep the final" still deleted the final; "the robot project" returned an empty plan	Independent AI review, round 2	Missing edge cases	Two US-03 tests red, then keep-clauses and a "which projects?" question
14	A plain request with "but keep my capstone"	Same review	Missing edge case	**Code first, test right after** (US-03 last test); disclosed
15	Unused `const TODO` lines	Same review	Dead code in tests	Removed 2 in round 3, but **5 more were missed** and our log wrongly said all were gone; the round-3 review caught it, and all 7 are now removed
16	"I'm not done with CS 3140", "still need the final", "Keep X, I'm done with the rest", and "before I leave for break" were misread	Independent AI review, round 3	**Bugs**	Test red, then clause splitting, negation, and exact names win over course codes
17	Answering "Which projects?" with one project also planned its sibling	Same review	**Bug**	Test red, then the answer names exactly that project
18	The scanned folder itself was offered as a project	Same review	**Bug** (REG-5)	Test red, then excluded
19	US-01's Xcode/Docker/ML keys were asserted to exist but couldn't fail	Same review	Test that couldn't fail	New todo test that needs real totals; US-01 marked partial

Known limits:

- The secret check is cautious: a `node_modules` that ships test certificates (`.pem`) is refused.
- Measured FREED can be off if another app writes to the disk during a run. Tests use a fake disk.
- The tests need `git`. CI runs on macOS because the malware checks call `codesign` and `xattr`.
- Only 5 tests call the real Claude API, and they've run once (5/5). The model's output varies between runs, so a pass is evidence, not a guarantee. They aren't in CI, because CI has no API key.

6. Next stories, in sprint order

1. Record the demo on Claude as the backup video, and rerun `npm run test:live` before class (5/5 on Nov 4). (`alpha-review` was merged in #32; #2 was reopened because US-01 is partial.)
2. US-04 protect projects.
3. US-08 Xcode, US-09 Docker, US-10 ML caches (which also completes US-01), US-16 storage watch, US-21 Go Pro.

Stage & Next Steps

Where the Alpha stands, and the roadmap to Beta

Where we are

Issues closed	Story points done	Must stories closed	Real testers on record
18 of 29	**49 of 99**	**17 of 24**	**0**
62%; US-01 is partial	Must stories: 47 of 78 points	7 Must stories left	Our first Beta milestone

Alpha milestones

Milestone	Evidence	Status
Requirements broken into stories	29 stories with acceptance criteria, all GitHub issues (#2–#30)	✅ Done
Tests written before code	`c1c337d` adds tests for all 23 open issues; red run saved	✅ Done
Rebuild proof and real deletion	`d2f885d`, `454d384`; FREED is now the measured change on disk	✅ Done
Clean by intent	US-03: "I'm done with CS 3140" maps to the right projects	✅ Done
Get it back	One-action undo, one-click rebuild, macOS Trash (US-13, US-14, US-15)	✅ Done
CI and coverage	`npm test` on every push; 95.9% backend line coverage	✅ Done
Real Claude tested	LIVE-1…5 passed 5/5 on Nov 4 (one recorded run)	✅ Done
Real-user validation	No session notes or logs on record (Activity 1)	❌ Not done

Still open: US-01 (partial), US-04, US-08, US-09, US-10, US-16 and US-21 are Must stories, and all seven are in the Beta plan. US-22 (Should) and US-17, US-20 and US-23 (Could) aren't scheduled for Beta.

Beta roadmap

Proposed; team to confirm. The remaining Must work is 26 new points (US-04 3, US-08 5, US-09 5, US-10 5, US-16 5, US-21 3), and finishing US-08 to US-10 also completes US-01. Our plan assumes about 20 points a week, so four weeks leaves room for the real risk, which is recruiting and running testers.

Week of	Work	Done means
Nov 9	**Real MVP validation.** Commit the test plan and KPI definitions first, then run sessions on the MVP build (`8a3e506`). Also US-04, and rerun live Claude tests	At least 8 real testers (5 CS), every KPI traces to a session sheet or exported log in the repo, and Activity 1 and its decision are redone from them
Nov 16	US-08 Xcode, US-09 Docker. Teammates start reviewing tests	7 todo tests now pass; Xcode Archives and Docker volumes are proven NEVER
Nov 23	US-10 ML caches, US-16 storage watch	7 todo tests now pass; US-01 is closed; alert fires at 85% full
Nov 30	US-21 Go Pro and rebuild logging. Recruit 20 CS students from outside our friend group, and start the pivot test	All Must stories closed (24 of 24); 20 testers enrolled; logs export correctly
30 days after start	Pivot test read-out	Every KPI below reported with its sample size, and a written decision

If real sessions don't point to developer clutter, we stop before Sprint 4 work and revisit the target. The pivot is a bet until then.

Measurable goals for the Beta

Product (pivot test, from Activity 2 §7). 20 CS students from outside our friend group, unsupervised.

KPI	Target
Testers with at least 10 GB of rebuildable clutter	≥70%
First-session approval of a plan freeing at least 10 GB	≥60%
Real space freed, measured on disk (median)	≥10 GB
One-click rebuilds that work	≥95%
Repeat use within 30 days	≥40%
Unrecoverable deletions	0
"Go Pro" fake-door clicks / pre-orders	≥15% / ≥5%
Head-to-head with CleanMyMac's purge *(proposed)*	≥50% prefer ours; none lose work

Engineering.

Metric	Today	Beta target
Must stories closed	17 of 24	24 of 24
Must-story todo tests	19	0
Runnable pass rate	100%	100%
Backend line coverage	95.9%	≥95%
Live Claude tests	5/5, one run	5/5 on 3 runs in a row
Test files reviewed by a teammate	0	All, logged in `tdd/TEAM_REVIEW.md`

Appendix · The Alpha running

Captured Nov 5, 2026 from `main` at `787454f` , offline planner

The request "I'm done with CS 3140 and the hackathon. Free up space." on the `dev-demo` folder. Try 1 proposes the hackathon project too, and the validator rejects `hoohacks-2026` for uncommitted work (Safety Log, bottom right). Try 2 passes with the two CS 3140 folders, each showing its rebuild command.

